TUGAS 03

Bab 3 — Docker Network, Volume, Bind Mount, tmpfs, dan Compose

Matakuliah Workshop DevOps

Dosen Pengampu : Bapak Dr.Ferry Astika Saputra,ST, M.Sc



Oleh :

Nama : Toimul Setyo Andri

NRP : 3126640021

Kelas : D4 LJ TI kelas B

Politeknik Elektronika Negeri Surbaya

Departemen Teknik Informatika dan Komputer

Program Studi Teknik Informatika

September 2026

Langkah 1: Buat Direktori dan Struktur Proyek

1.1

```
toimul@DESKTOP-2OIAP1F:~$ mkdir -p ~/docker-lab/bab-3/app ~/docker-lab/bab-3/html
toimul@DESKTOP-2OIAP1F:~$ cd ~/docker-lab/bab-3
toimul@DESKTOP-2OIAP1F:~/docker-lab/bab-3$
```

Langkah 2: Buat Komponen Application (Flask App)

2.1 app/requirements.txt

```
toimul@DESKTOP-2OIAP1F:~/docker-lab/bab-3$ cat << 'EOF' > app/requirements.txt
> Flask==3.0.0
> psycopg2-binary==2.9.9
> EOF
```

2.2 app/app.py

```
toimul@DESKTOP-2OIAP1F:~/docker-lab/bab-3$ cat << 'EOF' > app/app.py
> import os
import psycopg2
from flask import Flask, jsonify

app = Flask(__name__)

def get_db_connection():
    conn = psycopg2.connect(
        host=os.environ.get('DB_HOST'),
        database=os.environ.get('DB_NAME'),
        user=os.environ.get('DB_USER'),
        password=os.environ.get('DB_PASS')
    )
    return conn

@app.route('/api')
def index():
    try:
        conn = get_db_connection()
        cur = conn.cursor()
        cur.execute('SELECT version();')
        db_version = cur.fetchone()
        cur.close()
        conn.close()
        return jsonify({"status": "success", "db_version": db_version[0]})
    except Exception as e:
        return jsonify({"status": "error", "message": str(e)}), 500

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
> EOF
```

2.3 app/Dockerfile

```
toimul@DESKTOP-2OIAP1F:~/docker-lab/bab-3$ cat << 'EOF' > app/Dockerfile
> FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
EOF
```

2.4 Verifikasi

```
toimul@DESKTOP-2OIAP1F:~/docker-lab/bab-3$ ls -la app/
total 20
drwxr-xr-x 2 toimul toimul 4096 Sep 11 12:52 .
drwxr-xr-x 4 toimul toimul 4096 Sep 11 12:46 ..
-rw-r--r-- 1 toimul toimul  156 Sep 11 12:52 Dockerfile
-rw-r--r-- 1 toimul toimul  786 Sep 11 12:51 app.py
-rw-r--r-- 1 toimul toimul   36 Sep 11 12:49 requirements.txt
```

Langkah 3: Buat Konfigurasi Nginx dan HTML

3.1 nginx.conf

```
toimul@DESKTOP-2OIAP1F:~/docker-lab/bab-3$ cat << 'EOF' > nginx.conf
> server {
    listen 80;

    location / {
        root /usr/share/nginx/html;
        index index.html;
    }

    location /api {
        proxy_pass http://app:5000/api;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
EOF
```

3.2 html/index.html

```
toimul@DESKTOP-2OIAP1F:~/docker-lab/bab-3$ echo "<h1>Welcome to Nginx-Flask-Postgres Toi App</h1>" > html/index.html
toimul@DESKTOP-2OIAP1F:~/docker-lab/bab-3$
```

3.3 Verifikasi

```
toimul@DESKTOP-2OIAP1F:~/docker-lab/bab-3$ ls -l nginx.conf
-rw-r--r-- 1 toimul toimul 265 Sep 11 12:55 nginx.conf
toimul@DESKTOP-2OIAP1F:~/docker-lab/bab-3$
```

Langkah 4: Buat File docker-compose.yml

4.1 Membuat file compose

```
toimul@DESKTOP-2OIAP1F:~/docker-lab/bab-3$ cat << 'EOF' > docker-compose.yml
>
> services:
  web:
    image: nginx:alpine
    ports:
      - "8080:80"
    volumes:
      - ./html:/usr/share/nginx/html:ro
      - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
    networks: [frontend]
    depends_on: [app]

  app:
    build: ./app
    environment:
      DB_HOST: db
      DB_NAME: labdb
      DB_USER: labuser
      DB_PASS: labpass123
    networks: [frontend, backend]
    depends_on:
      db:
        condition: service_healthy
```

```
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: labdb
      POSTGRES_USER: labuser
      POSTGRES_PASSWORD: labpass123
    volumes:
      - pg-data:/var/lib/postgresql/data
    networks: [backend]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U labuser -d labdb"]
      interval: 5s
      timeout: 5s
      retries: 5

volumes:
  pg-data:

networks:
  frontend:
  backend:
EOF
```

4.2 Verifikasi: menjalankan docker compose config

```
toimul@DESKTOP-2OIAP1F:~/docker-lab/bab-3$ docker compose config
name: bab-3
services:
  app:
    build:
      context: /home/toimul/docker-lab/bab-3/app
      dockerfile: Dockerfile
    depends_on:
      db:
        condition: service_healthy
        required: true
    environment:
      DB_HOST: db
      DB_NAME: labdb
      DB_PASS: labpass123
      DB_USER: labuser
    networks:
      backend: null
      frontend: null
  db:
    environment:
      POSTGRES_DB: labdb
      POSTGRES_PASSWORD: labpass123
      POSTGRES_USER: labuser
    healthcheck:
      test:
        - CMD-SHELL
        - pg_isready -U labuser -d labdb
      timeout: 5s
      interval: 5s
      retries: 5
    image: postgres:16-alpine
    networks:
```

```
      backend: null
    volumes:
      - type: volume
        source: pg-data
        target: /var/lib/postgresql/data
        volume: {}
  web:
    depends_on:
      app:
        condition: service_started
        required: true
    image: nginx:alpine
    networks:
      frontend: null
    ports:
      - mode: ingress
        target: 80
        published: "8080"
        protocol: tcp
```

```
    volumes:
      - type: bind
        source: /home/toimul/docker-lab/bab-3/html
        target: /usr/share/nginx/html
        read_only: true
        bind: {}
      - type: bind
        source: /home/toimul/docker-lab/bab-3/nginx.conf
        target: /etc/nginx/conf.d/default.conf
        read_only: true
        bind: {}
networks:
  backend:
    name: bab-3_backend
  frontend:
    name: bab-3_frontend
volumes:
  pg-data:
    name: bab-3_pg-data
```

Langkah 5: Jalankan dan Verifikasi Stack Multi-Container

5.1 Menjalankan Compose stack di background

```
toimul@DESKTOP-2OIAP1F:~/docker-lab/bab-3$ docker compose up -d --build
[+] up 13/13
 ✔Image postgres:16-alpine Pulled
[+] Building 46.4s (13/13) FINISHED
 => [internal] load local bake definitions
 => => reading from stdin 502B
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 195B
 => [internal] load metadata for docker.io/library/python:3.11-slim
 => [auth] library/python:pull token for registry-1.docker.io
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [1/5] FROM docker.io/library/python:3.11-slim@sha256:9534e5a8e315485d4061ed659af0fd78a284c015f9b73661b41d6
 => => resolve docker.io/library/python:3.11-slim@sha256:9534e5a8e315485d4061ed659af0fd78a284c015f9b73661b41d6
 => => sha256:f9efa1b83d065a7c5a3041582875b75d63c4ab3d4413514de8e413b51f90e91a 249B / 249B
 => => sha256:db840d086b65cf73e0feea3bd0cf11063818114e466846ae3f2b2fb1a8c2b143 14.45MB / 14.45MB
 => => sha256:6310eb16bf4251731feab01e8f633bf5e2d75a657ccad97f420b1f83cce457be 29.79MB / 29.79MB
 => => sha256:3678bb828654fcc3752f7a5eb63c5accfa00c97047252f25405285550d74a3fd 4.27MB / 4.27MB
```

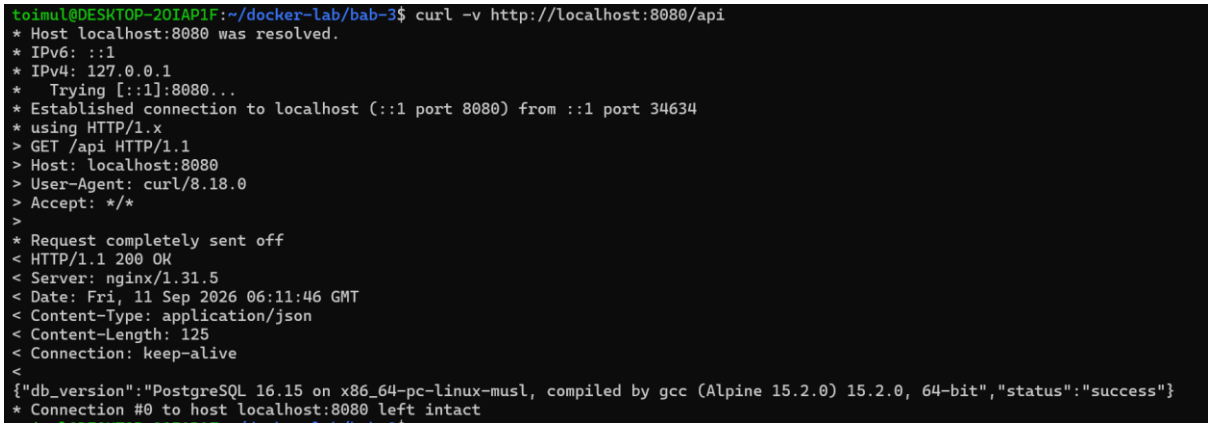
5.2 Tunggu beberapa detik (healthcheck PostgreSQL akan memverifikasi kesiapan DB), lalu periksa status service dan pastikan Pastikan kontainer db berstatus healthy, serta web dan app berstatus running.

```
toimul@DESKTOP-2OIAP1F:~/docker-lab/bab-3$ docker compose ps
NAME          IMAGE              COMMAND                  SERVICE   CREATED              STATUS                        PORTS
bab-3-app-1   bab-3-app          "python app.py"          app       About a minute ago   Up About a minute             5000/tcp
bab-3-db-1    postgres:16-alpine "docker-entrypoint.s…"   db        About a minute ago   Up About a minute (healthy)   5432/tcp
bab-3-web-1   nginx:alpine       "/docker-entrypoint.…"   web       About a minute ago   Up About a minute             0.0.0.0:8080
->80/tcp, [::]:8080->80/tcp
```

5.3 Verifikasi Web Static (Nginx): Output harus menampilkan tag *<h1>Welcome to Nginx-Flask-Postgres Toi App</h1>*

```
toimul@DESKTOP-2OIAP1F:~/docker-lab/bab-3$ curl -v http://localhost:8080
* Host localhost:8080 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8080...
* Established connection to localhost (::1 port 8080) from ::1 port 60728
* using HTTP/1.x
> GET / HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/8.18.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Server: nginx/1.31.5
< Date: Fri, 11 Sep 2026 06:09:56 GMT
< Content-Type: text/html
< Content-Length: 49
< Last-Modified: Fri, 11 Sep 2026 05:57:26 GMT
< Connection: keep-alive
< ETag: "6aa39846-31"
< Accept-Ranges: bytes
<
<h1>Welcome to Nginx-Flask-Postgres Toi App</h1>
* Connection #0 to host localhost:8080 left intact
```

5.4 Verifikasi API Connection (Nginx -> Flask -> PostgreSQL): Output harus mengembalikan balasan JSON dengan status success dan rincian db_version PostgreSQL 16.



Langkah 6 Kesimpulan

Praktikum ini berhasil membuktikan bagaimana **Docker Compose** menyatukan dan mengorkestrasi tiga komponen utama aplikasi web (*Nginx, Flask, PostgreSQL*) ke dalam satu ekosistem yang terisolasi dan dapat diandalkan:

1. **Segmentasi Jaringan (Network Isolation)**

- **Network Frontend:** Menghubungkan Nginx dan Flask. Nginx bertindak sebagai satu-satunya pintu masuk (*reverse proxy*) yang mengekspos port ke host (8080:80).
- **Network Backend:** Menghubungkan Flask dan PostgreSQL. Database dibuat sepenuhnya privat tanpa memublikasikan port ke host, sehingga terlindungi dari akses luar.

2. **Resolusi DNS Internal**

- Antar-container saling berkomunikasi menggunakan nama service (misalnya, Flask menghubungi db dan Nginx meneruskan request ke app:5000) tanpa perlu bergantung pada alamat IP dinamis.

3. **Orkestrasi dan Healthcheck**

- Penggunaan depends_on yang dikombinasikan dengan condition: service_healthy memastikan Flask tidak akan dinyalakan sebelum PostgreSQL benar-benar siap menerima koneksi.

4. **Persistensi Data**

- Menggunakan *Named Volume* (pg-data) untuk PostgreSQL sehingga data database tetap aman dan tidak hilang meskipun container dihentikan atau dihapus.